

Bridging Correctness and Code Quality in LLM Generation: A Two-Stage Generate-and-Refine Pipeline

Nouman Munib

Unaffiliated

Pakistan

noumanmunib27@gmail.com

Abstract

Large language models (LLMs) can generate code that passes test cases, but the generated code may still show maintainability issues or other quality concerns. This paper proposes a two-stage Generate-and-Refine pipeline that separates correctness-oriented generation from subsequent quality-oriented refinement. In Stage 1, few-shot prompting is used to obtain solutions that pass the provided test suites. In Stage 2, we apply and compare three refinement strategies: specialized LLM prompting, deterministic rule-based refactoring, and retrieval-augmented exemplar-guided refinement. To limit regressions, each refined candidate is re-validated against the original test suite and discarded if correctness degrades. We evaluate the pipeline on the APPS dataset using correctness measures and automated code quality metrics, including SonarQube indicators and maintainability indices. The study reports which refinement strategies improve code quality while preserving functional behavior.

CCS Concepts

• **Software and its engineering** → **Automatic programming**; **Source code generation**; **Software development methods**; **Maintaining software**; • **Computing methodologies** → **Natural language processing**.

Keywords

Code Generation; Large Language Models; Code Quality; Automated Refactoring; Few-Shot Prompting

ACM Reference Format:

Nouman Munib. 2026. Bridging Correctness and Code Quality in LLM Generation: A Two-Stage Generate-and-Refine Pipeline. In *5th International Workshop on Natural Language-based Software Engineering (NLBSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3786164.3788446>

1 Introduction

Large language models (LLMs) are increasingly applied to program synthesis and code completion. Many models can produce solutions that satisfy test cases. Prior work reports cases where generated code passes tests but exhibits maintainability issues, inefficient patterns, or other concerns [1–3].

This paper proposes a two-stage Generate-and-Refine pipeline. Stage 1 focuses on producing functionally correct solutions via few-shot prompting [6]. Stage 2 applies targeted refinement strategies to those correct solutions. Each refinement is followed by re-execution of the test suite. Only refinements that preserve correctness are accepted. The design reduces the risk of functional regressions while allowing quality-oriented changes.

In this work, *functional correctness* denotes that a program meets its specified behavior as verified by the provided test harness. *Code quality* denotes secondary properties such as maintainability, readability, and the presence of code smells, as measured by automated tools. Correctness is a necessary condition for quality but does not subsume these other properties. The pipeline treats correctness as a prerequisite and evaluates whether automated refinement can raise other quality measures without changing program behavior.

Recent work has catalogued defects and inefficiencies in LLM-generated code, such as [4]. Those studies highlight the problem; our work differs by proposing and empirically comparing refinement strategies that preserve correctness while targeting measurable improvements in quality.

2 Proposed Approach

The pipeline follows two stages.

2.1 Stage 1: Generation for correctness

We employ few-shot prompting with current LLMs to produce initial candidate solutions [6]. The generation step aims to obtain solutions that pass the provided test suites. For each problem we generate a set of candidates and validate them using the problem's test harness. Only candidates that pass the harness are retained for refinement.

2.2 Stage 2: Refinement for quality

We evaluate three distinct refinement strategies to passing solutions. The first uses targeted LLM prompts to request refactoring or idiomatic rewrites with explicit objectives such as reducing complexity or improving naming. The second uses deterministic rule-based refactoring (method extraction, variable renaming, small structural changes) implemented as reproducible program transformations. The third employs retrieval-augmented generation [7]: for each target problem the system retrieves exemplars of high-quality implementations from a curated corpus and uses these exemplars to guide a context-aware refinement prompt. Each refinement is followed by re-execution of the problem's test harness; any refined version that fails is discarded. This validation step preserves functional behavior while allowing quality-oriented changes to be explored.



This work is licensed under a Creative Commons Attribution 4.0 International License. *NLBSE '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2396-4/2026/04

<https://doi.org/10.1145/3786164.3788446>

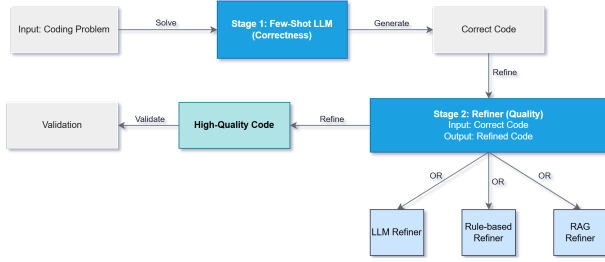


Figure 1: Two-stage pipeline showing input problem, few-shot LLM generation, correctness validation, and three candidate refinement approaches (LLM, rule-based, and RAG) leading to higher-quality code.

3 Evaluation and Expected Contributions

3.1 Concrete evaluation plan

We use Python problems from the APPS dataset [5]. For each problem, we begin by generating a set of candidate solutions using few-shot prompting [6]. The generated candidates are then validated using the problem’s original test harness, and only those that pass all tests are retained. Next, each of the three refinement strategies is applied to the passing solutions. After each refinement, we re-run the test harness to verify functional correctness is preserved. Finally, we evaluate the quality of each refined solution using SonarQube metrics [8], including bug detection, code smells, maintainability index, and cyclomatic complexity. We report Pass@1 before and after refinement and the deltas in the quality metrics. Exact prompt templates, candidate counts, rule sets for the rule-based refiner, retrieval corpora for RAG, and SonarQube configuration will be provided in the replication materials.

3.2 Metrics and Selection Rule

We use Pass@1 to measure functional correctness. For code quality, we use SonarQube metrics: bugs, code smells, maintainability index, and cyclomatic complexity. We also report readability scores and percentage of duplicated code.

For each strategy, we compute the difference in metrics compared to the initial passing solution. If a strategy reduces Pass@1 significantly ($\alpha = 0.05$), it is excluded from comparison. The rest are ranked by a weighted quality score combining the metric deltas. We report both overall and per-metric results. Full details and weights are included in the replication materials.

3.3 Expected contributions

The study provides an empirical comparison of targeted refinement strategies applied to functionally correct LLM output. It offers practical guidance on applying refiners while protecting correctness. It presents a modular pipeline that can be adapted to other quality measures and programming domains.

4 Threats to Validity

The study relies on automated test suites and static quality metrics. Test suites in APPS may not capture full functional intent, and static

tools (SonarQube) approximate human judgment. These factors limit external validity. To mitigate these threats we re-run tests after every refinement and discard regressions. We report numerical metric changes and representative code examples. We restrict claims to the APPS dataset and note that validation on large, real-world systems is left to future work.

5 Future Work

A follow-up study should include human judgment. We plan a developer-centric evaluation in which experienced programmers rate comprehension, debugging confidence, and change resilience. We will also extend evaluation to additional languages and to code drawn from real projects. Such work will test whether automated metric gains align with human assessments.

6 Conclusion

We present a two-stage Generate-and-Refine pipeline. The pipeline treats correctness as a prerequisite and applies quality-oriented refinements only to functionally correct solutions. Each refinement is validated and discarded if it regresses. The planned evaluation on APPS plus SonarQube metrics will identify which refinement strategies improve code quality while preserving functional behavior.

Acknowledgments

The author acknowledges the use of automated writing tools for clarity and structure. All research decisions and content are the author’s own.

References

- [1] Molison, A. S., Moraes, M., Melo, G., et al. 2025. Is LLM-Generated Code More Maintainable & Reliable Than Human-Written Code? *arXiv preprint arXiv:2508.00700v1 [cs.SE]*.
- [2] Jiang, N., Lutellier, T., and Tan, S. H. 2024. What’s Wrong With Your Code Generated By Large Language Models? In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [3] Figueiredo, R., Alves, V., and Oliveira, S. 2025. Evaluating The Effectiveness Of LLMs In Fixing Maintainability Issues. In *Proceedings of the International Conference on Software Analysis, Evolution and Reengineering (SANER)*.
- [4] Abbassi, A. A., Da Silva, L., Nikanjam, A., and Khomh, F. 2025. Unveiling inefficiencies in LLM-generated code: toward a comprehensive taxonomy. *arXiv preprint arXiv:2503.06327*.
- [5] Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., and Steinhardt, J. 2021. Measuring Coding Challenge Competence With APPS. In *Advances in Neural Information Processing Systems (NeurIPS)*, 7566–7583.
- [6] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Nee-lakantan, A., Shyam, P., Sastry, G., Askell, A., et al. 2020. Language Models Are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 1877–1901.
- [7] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., et al. 2020. Retrieval-Augmented Generation For Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 9459–9474.
- [8] SonarSource SA. 2024. SonarQube: Code Quality and Security. <https://www.sonarqube.org/>